Hook It: System Design Document

Course: CEN 5064 Software Design — Fall 2026

Institution: Florida International University

Authors: Dionny Dinza

Date: Oct 6 2026, Revised: Nov 2, 2026

Table of Contents

HookIt: System Design Document 1

1. System Scope 3

2. N-Tier Architecture 4

3. System Diagrams 5

3.1 C4 Context Diagram 5

3.2 C4 Container Diagram 6

3.3 Sequence Diagram 7

3.4 UML Class Diagram 8

4. Plan for the Second Half of the Semester 9

1. System Scope

HookIt is a native iOS application designed for recreational anglers to digitally log their catches while automatically ensuring compliance with local fishing regulations. To maintain a strict scope and clear architectural tiers, the system focuses on four core features:

1. **AI Fish Identification:** An external API integration that analyzes an uploaded photo to identify the species.
2. **Catch Logger:** A secure interface where users record their harvest details (species, length, date).
3. **Live Fishing Conditions:** An integration that fetches real-time tide data from the NOAA API based on the user's current GPS coordinates.
4. **Compliance Engine (Domain Rule):** A validation layer that automatically cross-references every logged catch against a built-in static database of regional size limits and open seasons to instantly warn the user if a fish is undersized or out of season.

2. N-Tier Architecture

HookIt enforces separation of concerns through a 4-tier architectural model, ensuring the codebase remains modular and testable.

Presentation Tier: Built natively for iOS using SwiftUI. Core components include the AppShellView navigation container, HomeView, IdentifyItView, and AddCatchView. While primarily responsible for rendering the UI, views like AddCatchView act as controllers that directly call the Domain tier (RegulationValidator) to check rules, evaluate the result, and call the Data tier to save the record.

Service Tier (Application Logic): Acts as the orchestrator routing data between the application and external APIs. Key components include TideService and WeatherService (managing geolocation and NOAA data retrieval), and GeminiService (handling AI image requests).

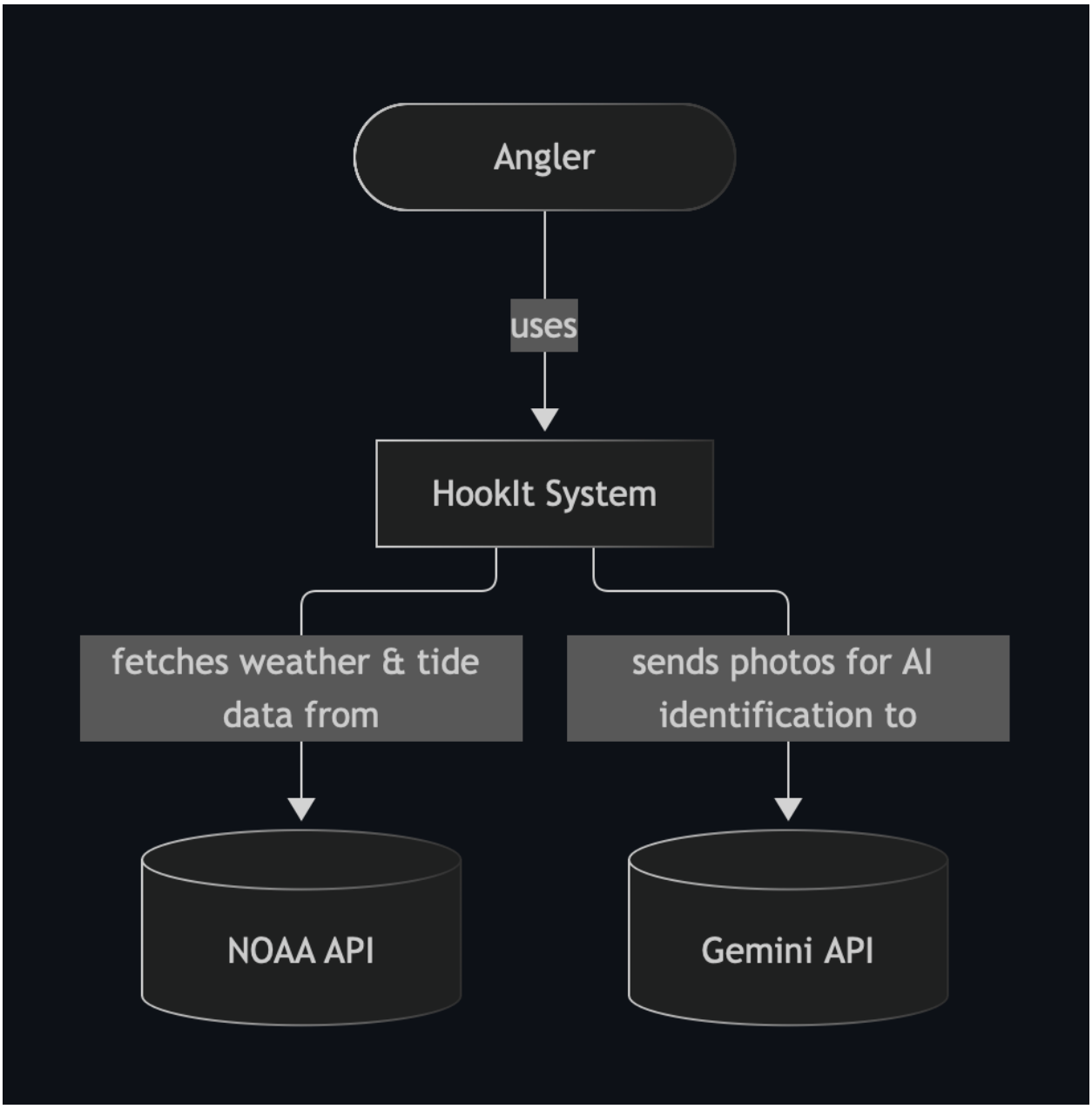
Domain Tier (Business Logic): Houses the core, immutable rules of the application. The primary engine is the RegulationValidator, which receives catch data (species name, length, and date) and calculates a ComplianceStatus based on Florida fishing laws. It is completely isolated from network and UI rendering concerns.

Data Tier: Manages all data models and persistence. It encapsulates static reference tables and the local catch history. The primary component here is CatchManager, which operates as a Singleton to safely serialize and persist CatchRecord arrays to the device's UserDefaults.

3. System Diagrams

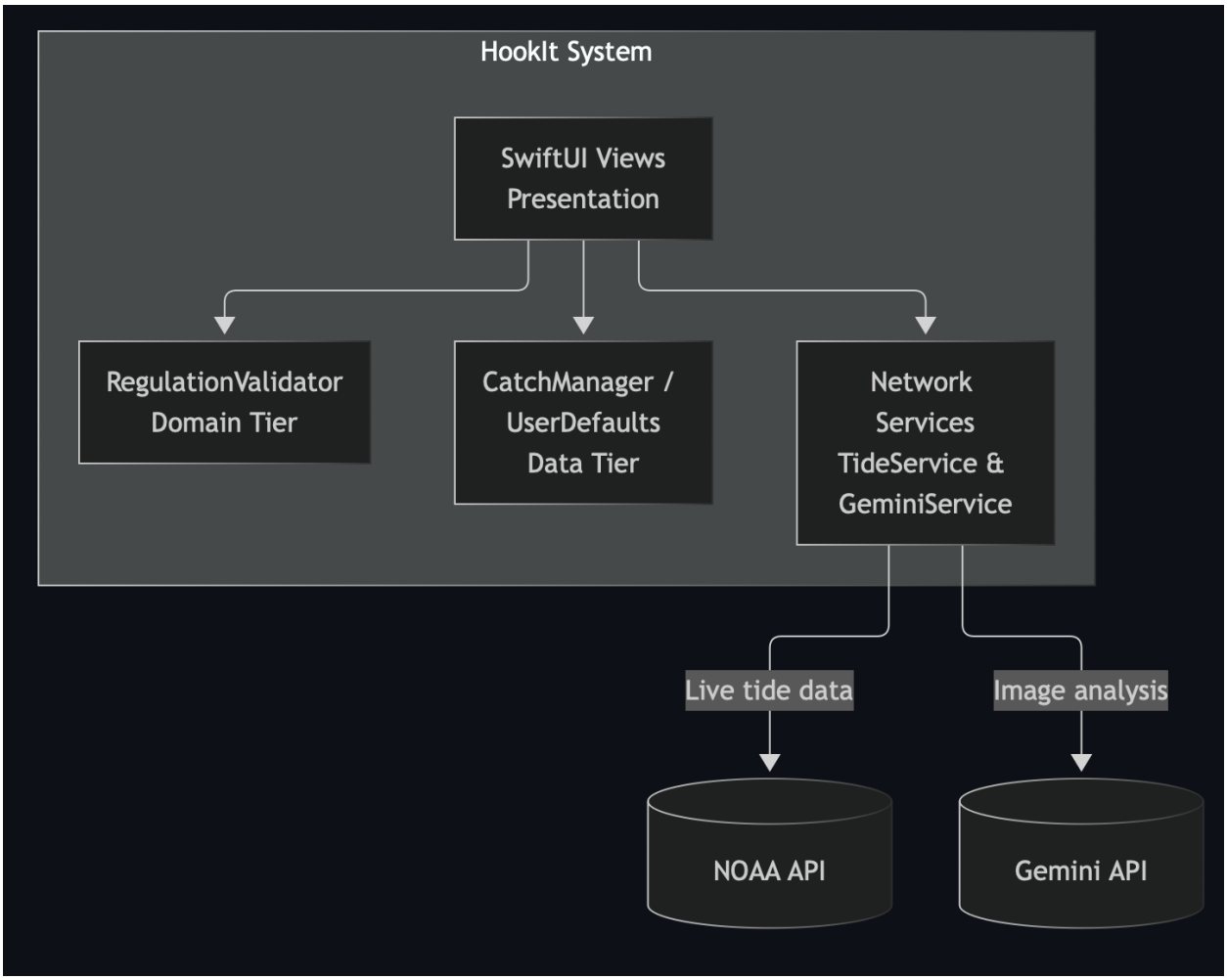
3.1 C4 Context Diagram

This diagram illustrates HookIt's high-level system boundaries, showing how the primary actor (the Angler) interacts with the application and how the system communicates with external dependencies (the NOAA API and Gemini Vision API).



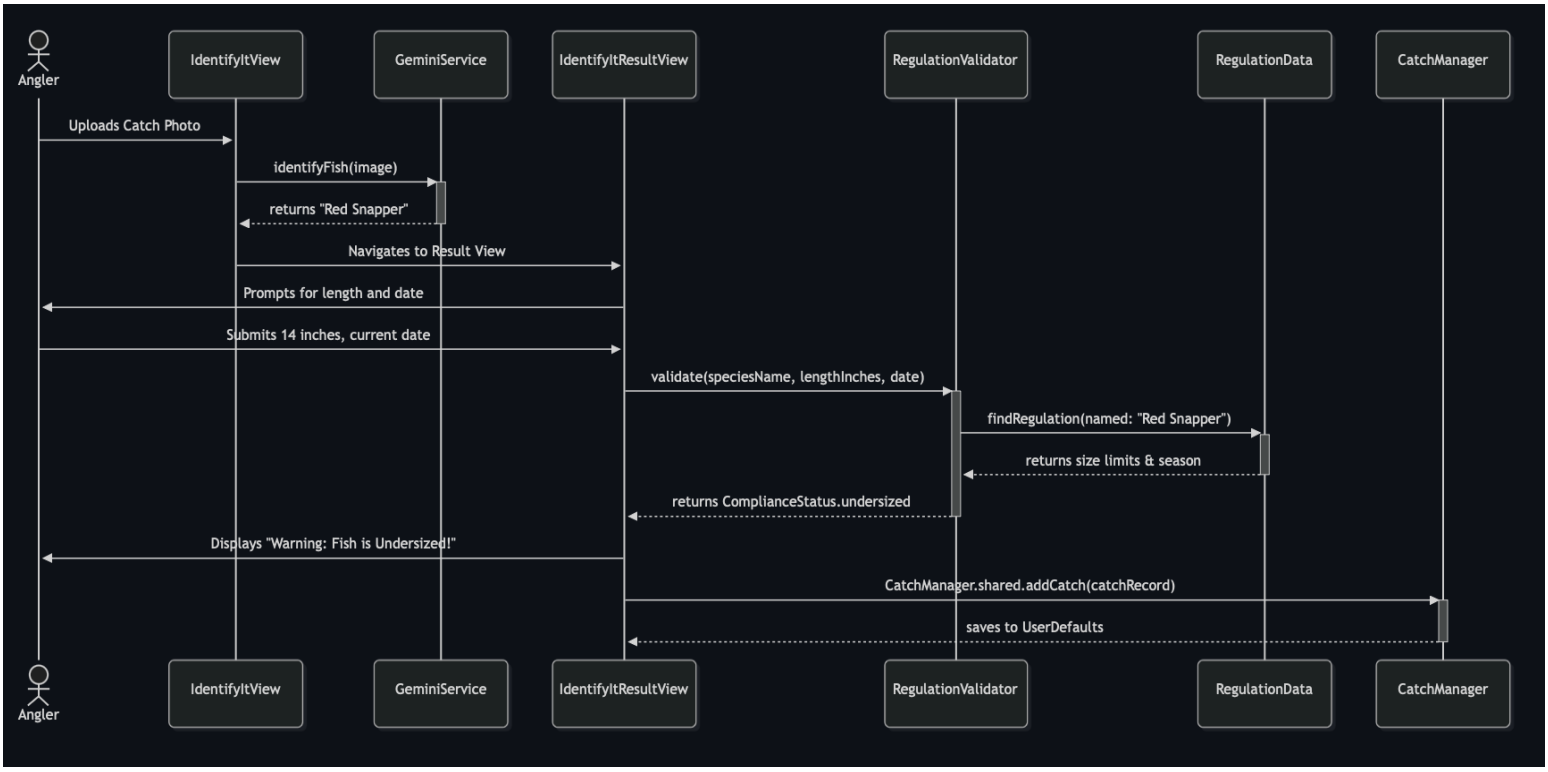
3.2 C4 Container Diagram

This diagram breaks down the highest level of the architecture into its primary executable containers. It highlights the native iOS application block and its direct data flow to the external APIs, establishing the system's runtime environment.



3.3 Sequence Diagram

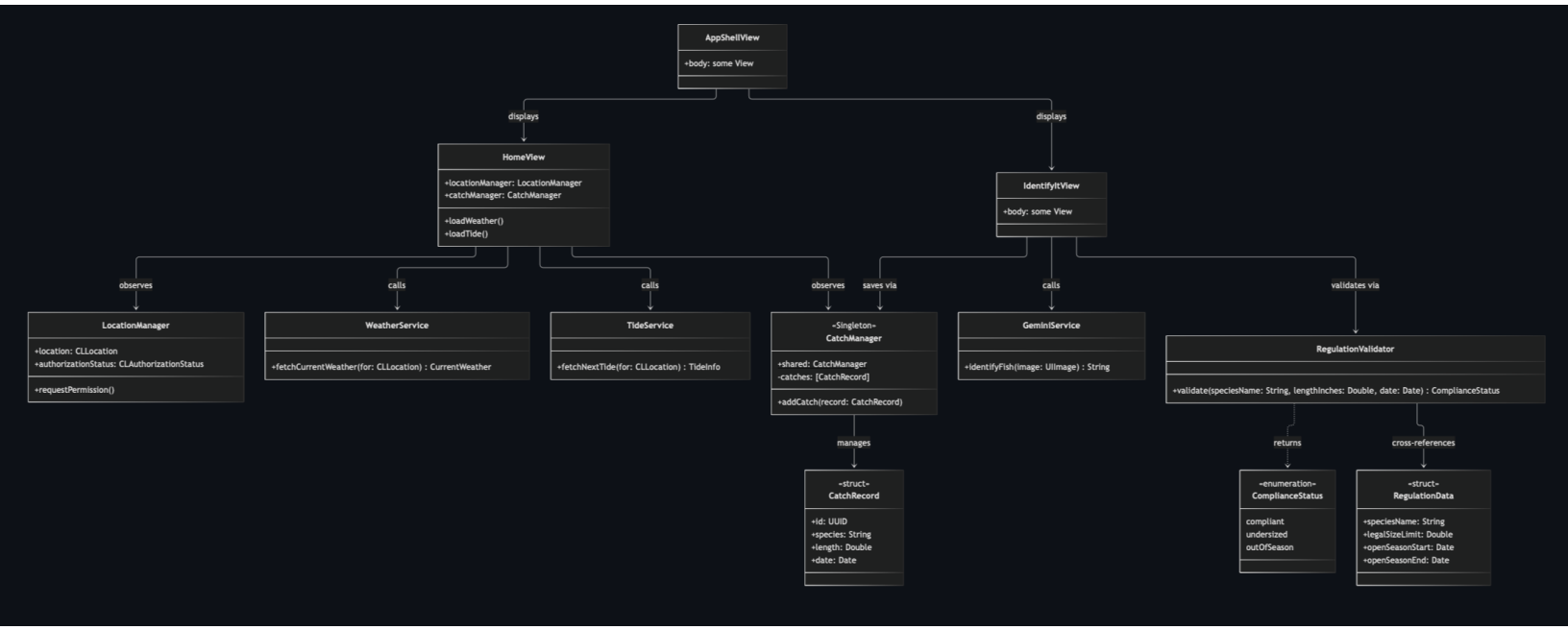
This sequence maps the step-by-step execution flow of the system's core use case: the user uploading a catch photo, the system requesting AI identification, navigating to the results view, and the domain tier validating that catch against local fishing regulations.



3.4 UML Class Diagram

This diagram details the specific Swift implementation of the system's 4-tier architecture. It accurately maps the views, state objects, services, and data models currently executing the application's core logic.

* UML Class Diagram is also on the repository’s README if hard to see. (https://github.com/ddinza/cen5064-project-dinza/blob/main/README.md)



4. Plan for the Second Half of the Semester

For the remainder of the semester, development will shift from building the core UI and mock-data workflows to stabilizing external API integrations, establishing persistent on-device storage, and expanding the core business logic. The roadmap focuses on four primary objectives:

Weeks 8–9: Stabilizing API & Integrating Peer Feedback

- **Focus:** Implement exponential backoff for `GeminiService` to handle 503 errors, and update `RegulationValidator` to enforce seasonal date ranges using the `catchDate` parameter (addressing midterm peer review feedback).
- **Risk:** External server outages block the core AI workflow, or complex date logic breaks existing species validation.
- **Mitigation:** Build a fallback manual entry UI for the API, and write isolated unit tests for the new date-range logic.

Weeks 10–12: Live Network & Local Persistence

- **Focus:** Replace simulated `Task.sleep` data with `URLSession` fetches for the NOAA API, and implement `SwiftData` for the `CatchManager` to ensure offline persistence.
- **Risk:** Data schema changes during development corrupt existing saved catches.
- **Mitigation:** Establish strict SwiftData model versioning and migration plans from the start.

Weeks 13–15: Advanced Domain Validation

- **Focus:** Expand the Compliance Engine to handle slot limits (fish between two specific sizes) and regional daily bag limits.
- **Risk:** Complex, overlapping state regulations cause edge-case validation bugs.
- **Mitigation:** Write isolated unit tests specifically covering boundary conditions (e.g., fish measuring exactly on the legal size limit).